

Use Case 4: Real-Time IoT Sensor Monitoring

Stream Processing with Kafka, PySpark Structured Streaming, and Snowflake

1. Overview

In this use case you will build a **real-time streaming data pipeline** that ingests IoT sensor readings from Apache Kafka, processes them with PySpark Structured Streaming, and persists the results into Snowflake. The pipeline performs three concurrent operations on the live data stream:

1. **Raw ingestion** -- every sensor reading is stored for historical analysis.
2. **Windowed aggregation** -- rolling 5-minute averages, minimums, and maximums are computed per sensor.
3. **Anomaly detection** -- readings that exceed safe operating thresholds trigger alerts.

This is a representative pattern used across manufacturing, logistics, and infrastructure monitoring.

2. Business Scenario

SmartFactory Inc. operates a modern manufacturing facility with **50 IoT sensors** deployed across factory floors, warehouses, and server rooms. Each sensor reports temperature, humidity, and atmospheric pressure every few seconds.

The operations team needs:

- A **live dashboard** showing current sensor health across all locations.
- **Rolling averages** (5-minute windows) to detect gradual environmental drift.
- **Immediate anomaly alerts** when temperature exceeds 80 F or humidity exceeds 70%, which could indicate equipment overheating or HVAC failure.
- All data stored in **Snowflake** for long-term trend analysis and compliance reporting.

Your assignment: Build the end-to-end streaming pipeline that satisfies these requirements.

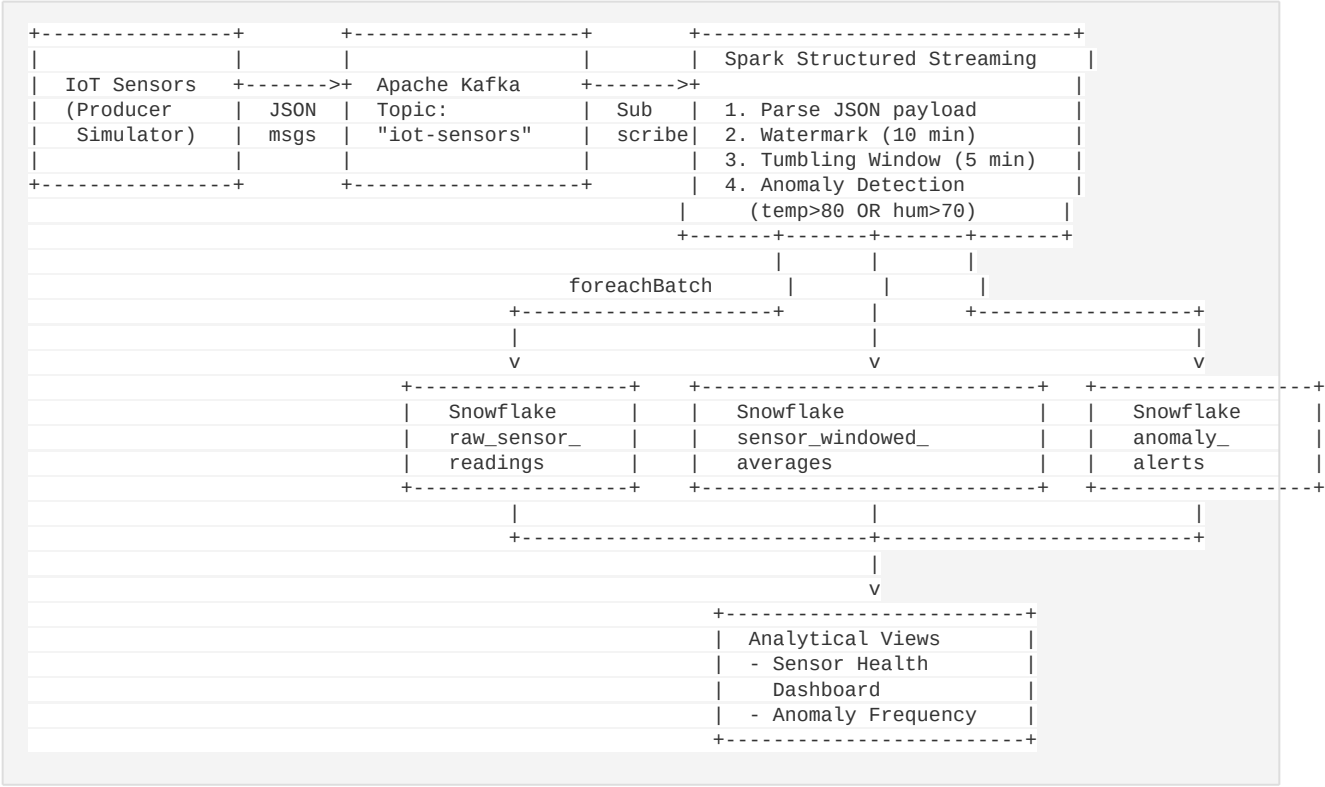
3. Learning Objectives

After completing this use case you will be able to:

#	Objective
1	Explain core Kafka concepts: topics, partitions, producers, consumers, offsets
2	Set up a local Kafka cluster using Docker Compose

3	Create a Kafka producer that generates simulated IoT data
4	Build a PySpark Structured Streaming application that reads from Kafka
5	Parse nested JSON payloads within a streaming DataFrame
6	Apply tumbling window aggregations on event-time columns
7	Implement threshold-based anomaly detection on streaming data
8	Use the <code>foreachBatch</code> sink pattern to write streaming results to Snowflake
9	Manage watermarks, checkpoints, and late-arriving data
10	Monitor and troubleshoot a running Structured Streaming query

4. Architecture Diagram



5. Dataset Description

5.1 JSON Message Schema

Every Kafka message has the following structure:

```
{
  "sensor_id": "SENSOR_01",
  "temperature": 72.5,
  "humidity": 45.2,
  "pressure": 1013.25,
  "location": "Factory_Floor_A",
  "timestamp": "2024-06-15T10:30:00Z"
}
```

Field	Type	Description
sensor_id	string	Unique identifier, SENSOR_01 through SENSOR_50
temperature	float	Degrees Fahrenheit. Normal range: 65 - 78. Anomaly: > 80
humidity	float	Relative humidity %. Normal range: 35 - 60. Anomaly: > 70
pressure	float	Atmospheric pressure in hPa. Normal range: 1010 - 1020
location	string	One of: Factory_Floor_A, Factory_Floor_B, Warehouse_1, Warehouse_2, Server_Room
timestamp	string	ISO 8601 UTC timestamp of the reading

5.2 Kafka Topic Structure

- **Topic name:** `iot-sensors`
- **Partitions:** 3 (data is partitioned by `sensor_id` as the message key)
- **Replication factor:** 1 (local development)
- **Message key:** `sensor_id` (ensures all readings from the same sensor go to the same partition)
- **Message value:** JSON string (the full reading object above)

5.3 Sample Data File

A static file with 20 sample records is provided at:

```
datasets/use_case_4/sample_sensor_data.json
```

This file uses JSON Lines format (one JSON object per line) and can be used for local testing before connecting to Kafka.

6. Tasks

Task 1: Start Kafka Using Docker Compose

Set up a local Kafka environment. Create a `docker-compose.yml` file (or use the one provided) with Zookeeper and Kafka services.

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Start the services:

```
docker-compose up -d
```

Verify Kafka is running:

```
docker-compose logs kafka | grep "started"
```

Create the topic:

```
docker exec -it <kafka-container> kafka-topics --create \
  --bootstrap-server localhost:9092 \
  --topic iot-sensors \
  --partitions 3 \
  --replication-factor 1
```

Task 2: Run the Kafka Producer to Simulate Sensor Data

The producer script is located at `streaming_utils/kafka_producer_iot.py`.

Install the dependency:

```
pip install kafka-python
```

Run the producer:

```
python streaming_utils/kafka_producer_iot.py \
  --num-sensors 10 \
  --interval 2 \
  --topic iot-sensors \
  --anomaly-probability 0.15
```

You should see output like:

```
[2024-06-15T10:30:00Z] SENSOR_01 | temp= 72.5 | hum= 45.2 | pres=1013.25 | Factory_Floor_A
[2024-06-15T10:30:02Z] SENSOR_06 | temp= 88.7 | hum= 72.1 | pres=1009.55 | Server_Room ** ANOMALY **
```

Leave this running in a separate terminal while you work on the streaming consumer.

Task 3: Set Up Snowflake Tables

Connect to Snowflake and execute the DDL statements to create the target tables. You need three tables:

- 1. `raw_sensor_readings` -- stores every individual reading from the sensors.
- 2. `sensor_windowed_averages` -- stores the 5-minute window aggregation results.
- 3. `anomaly_alerts` -- stores detected anomaly events.

The SQL setup script is provided in `solutions/use_case_4/snowflake_setup.sql`, but try writing the DDL yourself first based on the schema descriptions.

Schema for `raw_sensor_readings`:

Column	Type
sensor_id	VARCHAR(20)
temperature	FLOAT
humidity	FLOAT
pressure	FLOAT
location	VARCHAR(50)
reading_timestamp	TIMESTAMP_NTZ
ingested_at	TIMESTAMP_LTZ (default CURRENT_TIMESTAMP)

Schema for `sensor_windowed_averages`:

Column	Type
window_start	TIMESTAMP_NTZ
window_end	TIMESTAMP_NTZ
sensor_id	VARCHAR(20)

avg_temperature	FLOAT
avg_humidity	FLOAT
avg_pressure	FLOAT
min_temperature	FLOAT
max_temperature	FLOAT
min_humidity	FLOAT
max_humidity	FLOAT
min_pressure	FLOAT
max_pressure	FLOAT
reading_count	INTEGER
computed_at	TIMESTAMP_LTZ (default CURRENT_TIMESTAMP)

Schema for anomaly_alerts:

Column	Type
alert_id	INTEGER (AUTOINCREMENT)
sensor_id	VARCHAR(20)
metric_name	VARCHAR(20)
metric_value	FLOAT
threshold	FLOAT
location	VARCHAR(50)
reading_timestamp	TIMESTAMP_NTZ
detected_at	TIMESTAMP_LTZ (default CURRENT_TIMESTAMP)

Task 4: Create a Spark Structured Streaming App to Read from Kafka

Create a PySpark application that:

1. Initializes a `SparkSession` with Kafka and Snowflake connector packages.
2. Reads from the `iot-sensors` Kafka topic as a streaming `DataFrame`.
3. Configures `startingOffsets` as "latest" (or "earliest" for replaying history).

The initial streaming DataFrame from Kafka will have columns: key, value, topic, partition, offset, timestamp, timestampType. The value column contains the raw bytes of our JSON message.

```
raw_stream = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "iot-sensors")
    .option("startingOffsets", "latest")
    .load()
)
```

Task 5: Parse the JSON Payload from the Kafka Value

The Kafka value column is binary. You need to:

1. Cast it to a `STRING`.
2. Define a `StructType` schema matching the JSON fields.
3. Use `from_json()` to parse the string into structured columns.
4. Convert the timestamp string to a proper `TimestampType`.

```
from pyspark.sql.types import StructType, StructField, StringType, FloatType
from pyspark.sql.functions import from_json, col, to_timestamp

sensor_schema = StructType([
    StructField("sensor_id", StringType()),
    StructField("temperature", FloatType()),
    StructField("humidity", FloatType()),
    StructField("pressure", FloatType()),
    StructField("location", StringType()),
    StructField("timestamp", StringType()),
])
```

Parse and flatten:

```
parsed_stream = (
    raw_stream
    .selectExpr("CAST(value AS STRING) as json_str")
    .select(from_json(col("json_str"), sensor_schema).alias("data"))
    .select("data.*")
    .withColumn("event_time", to_timestamp(col("timestamp")))
)
```

Task 6: Implement 5-Minute Tumbling Window Aggregation

Using the parsed stream with event-time, apply a watermark and compute rolling aggregations:

1. Add a **watermark** of 10 minutes on `event_time` (to handle late data).
2. Group by a **5-minute tumbling window** on `event_time` and by `sensor_id`.
3. Compute: `avg`, `min`, `max` for temperature, humidity, and pressure.
4. Count the number of readings in each window.

Key Spark functions to use: - `withWatermark("event_time", "10 minutes")` - `window(col("event_time"), "5 minutes")`
- `F.avg()`, `F.min()`, `F.max()`, `F.count()`

Task 7: Implement Anomaly Detection

Create a filtered stream that captures readings where:

- `temperature > 80` **OR**
- `humidity > 70`

For each anomalous reading, construct an alert record with: - The `sensor_id` and `location` - Which metric triggered the alert (`metric_name`) - The actual value (`metric_value`) and the threshold it exceeded - The event timestamp

Hint: You may need to "explode" a single reading into multiple alert rows if both temperature AND humidity are anomalous.

Task 8: Write Raw Readings to Snowflake via `foreachBatch`

The `foreachBatch` sink pattern lets you treat each micro-batch as a static `DataFrame` and use standard batch write operations.

```
def write_to_snowflake(df, epoch_id, table_name):
    """Write a micro-batch DataFrame to a Snowflake table."""
    if df.count() == 0:
        return
    (
        df.write
        .format("snowflake")
        .options(**snowflake_options)
        .option("dbtable", table_name)
        .mode("append")
        .save()
    )
```

Apply it to the raw readings stream:

```
raw_query = (
    parsed_stream
    .writeStream
    .foreachBatch(lambda df, epoch_id: write_to_snowflake(df, epoch_id, "raw_sensor_readings"))
    .outputMode("append")
    .option("checkpointLocation", "/tmp/checkpoints/raw_readings")
    .start()
)
```

Task 9: Write Windowed Averages and Anomaly Alerts

Set up two additional streaming queries with separate `foreachBatch` sinks and checkpoint locations:

1. **Windowed averages** -- output mode must be `"update"` (windows are updated as new data arrives within the watermark).
2. **Anomaly alerts** -- output mode is `"append"`.

Each query needs its own unique checkpoint directory.

```
/tmp/checkpoints/windowed_averages
/tmp/checkpoints/anomaly_alerts
```

Task 10: Monitor the Streaming Queries and Verify Data in Snowflake

Once all three queries are running:

1. Check query status in Spark:

```
for q in spark.streams.active:
    print(f"Query: {q.name}, Status: {q.status}, Recent Progress: {q.recentProgress}")
```

1. Connect to Snowflake and verify data is flowing:

```
-- Check raw readings count
SELECT COUNT(*) FROM raw_sensor_readings;

-- Check latest windowed averages
SELECT * FROM sensor_windowed_averages
ORDER BY window_end DESC
LIMIT 10;

-- Check anomaly alerts
SELECT * FROM anomaly_alerts
ORDER BY detected_at DESC
LIMIT 20;
```

1. Observe how the counts grow over time as the producer continues sending data.

7. Expected Outputs

7.1 Raw Sensor Readings Table

After a few minutes of running, the `raw_sensor_readings` table should contain hundreds of rows:

SENSOR_ID	TEMPERATURE	HUMIDITY	PRESSURE	LOCATION	READING_TIMESTAMP
SENSOR_01	72.5	45.2	1013.25	Factory_Floor_A	2024-06-15 10:30:00
SENSOR_06	88.7	72.1	1009.55	Server_Room	2024-06-15 10:32:45
...					

7.2 Windowed Averages Table

One row per sensor per 5-minute window:

WINDOW_START	WINDOW_END	SENSOR_ID	AVG_TEMP	AVG_HUM	AVG PRES	COUNT
2024-06-15 10:30:00	2024-06-15 10:35:00	SENSOR_01	72.8	46.0	1013.12	15
2024-06-15 10:30:00	2024-06-15 10:35:00	SENSOR_06	89.5	71.9	1009.37	12
...						

7.3 Anomaly Alerts Table

Rows appear only when thresholds are breached:

ALERT_ID	SENSOR_ID	METRIC_NAME	METRIC_VALUE	THRESHOLD	LOCATION	DETECTED_AT
1	SENSOR_06	temperature	88.7	80.0	Server_Room	2024-06-15 10:32:45
2	SENSOR_06	humidity	72.1	70.0	Server_Room	2024-06-15 10:32:45
3	SENSOR_09	temperature	91.2	80.0	Server_Room	2024-06-15 10:34:00
...						

7.4 Continuous Query Console Output

While the streaming job is running, the Spark console (or Spark UI at <http://localhost:4040>) shows:

```
-----  
Batch: 42  
-----  
+-----+-----+-----+-----+-----+-----+  
| sensor_id|temperature|humidity|pressure |location          |event_time        |  
+-----+-----+-----+-----+-----+-----+  
|SENSOR_03 |      85.1 |    38.5 | 1010.75 |Server_Room       |2024-06-15 10:31:00|  
|SENSOR_01 |      73.1 |    46.8 | 1013.00 |Factory_Floor_A   |2024-06-15 10:35:00|  
+-----+-----+-----+-----+-----+-----+
```

8. Hints

Level 1 -- Gentle Nudge

- Kafka messages arrive as binary in the `value` column. Your first step is always `CAST(value AS STRING)`.
- The `from_json` function needs a schema. Define a `StructType` that exactly matches the JSON field names and types.
- For windowed aggregation, remember that the `window()` function creates a struct column with `start` and `end` fields. You will need to flatten these before writing to Snowflake.
- The `foreachBatch` function signature is `def my_func(df, epoch_id)` -- both parameters are required.

Level 2 -- More Specific Guidance

- When setting up the Snowflake write inside `foreachBatch`, skip empty batches with `if df.count() == 0: return` to avoid unnecessary Snowflake connections.
- For anomaly detection, consider using `F.when` and `F.array/F.explode` to create separate alert rows for temperature and humidity anomalies from a single reading. Alternatively, use `UNION` of two filtered streams.
- Make sure each streaming query has a **unique checkpoint location**. Sharing checkpoints between queries will cause errors.
- Use `"update"` output mode for the windowed query and `"append"` output mode for raw readings and anomaly alerts.

Level 3 -- Near-Solution

- For the windowed aggregation, after `groupBy` you need to flatten the window struct:

```
windowed_df = (  
  parsed_stream  
    .withWatermark("event_time", "10 minutes")  
    .groupBy(window(col("event_time"), "5 minutes"), col("sensor_id"))  
    .agg(  
      F.avg("temperature").alias("avg_temperature"),  
      F.min("temperature").alias("min_temperature"),  
      F.max("temperature").alias("max_temperature"),  
      # ... repeat for humidity, pressure ...  
      F.count("*").alias("reading_count"),  
    )  
    .select(  
      col("window.start").alias("window_start"),  
      col("window.end").alias("window_end"),  
      col("sensor_id"),
```

```

        # ... all aggregated columns ...
    )
)

```

- For the anomaly detection with separate rows per metric:

```

temp_anomalies = (
    parsed_stream.filter(col("temperature") > 80)
    .select(
        col("sensor_id"),
        F.lit("temperature").alias("metric_name"),
        col("temperature").alias("metric_value"),
        F.lit(80.0).alias("threshold"),
        col("location"),
        col("event_time").alias("reading_timestamp"),
    )
)

humidity_anomalies = (
    parsed_stream.filter(col("humidity") > 70)
    .select(
        col("sensor_id"),
        F.lit("humidity").alias("metric_name"),
        col("humidity").alias("metric_value"),
        F.lit(70.0).alias("threshold"),
        col("location"),
        col("event_time").alias("reading_timestamp"),
    )
)

all_anomalies = temp_anomalies.union(humidity_anomalies)

```

9. Validation Queries

Run these queries in Snowflake after the pipeline has been running for at least 5 minutes to validate correctness.

9.1 Data Freshness Check

```

-- The most recent reading should be within the last few minutes
SELECT MAX(reading_timestamp) AS latest_reading,
       DATEDIFF('second', MAX(reading_timestamp), CURRENT_TIMESTAMP()) AS lag_seconds
FROM raw_sensor_readings;

```

9.2 Per-Sensor Reading Count

```

-- Each sensor should have a roughly equal number of readings
SELECT sensor_id, COUNT(*) AS reading_count
FROM raw_sensor_readings
GROUP BY sensor_id
ORDER BY sensor_id;

```

9.3 Windowed Average Consistency

```

-- The reading_count in windowed averages should match the raw count for the same window
SELECT w.sensor_id, w.window_start, w.window_end, w.reading_count AS windowed_count,
       r.raw_count
FROM sensor_windowed_averages w
JOIN (
    SELECT sensor_id,
           DATE_TRUNC('MINUTE', reading_timestamp) AS minute_bucket,
           COUNT(*) AS raw_count

```

```

    FROM raw_sensor_readings
    GROUP BY sensor_id, minute_bucket
) r ON w.sensor_id = r.sensor_id
ORDER BY w.window_start DESC
LIMIT 20;

```

9.4 Anomaly Validation

```

-- Every anomaly alert should correspond to a raw reading that exceeds the threshold
SELECT a.alert_id, a.sensor_id, a.metric_name, a.metric_value, a.threshold,
       CASE
         WHEN a.metric_name = 'temperature' AND a.metric_value > a.threshold THEN 'VALID'
         WHEN a.metric_name = 'humidity' AND a.metric_value > a.threshold THEN 'VALID'
         ELSE 'INVALID'
       END AS validation_status
FROM anomaly_alerts a
ORDER BY a.detected_at DESC
LIMIT 20;

```

9.5 Location-Level Summary

```

-- Summary of sensor health by location
SELECT location,
       COUNT(DISTINCT sensor_id) AS sensor_count,
       ROUND(AVG(temperature), 1) AS avg_temp,
       ROUND(AVG(humidity), 1) AS avg_humidity,
       SUM(CASE WHEN temperature > 80 THEN 1 ELSE 0 END) AS high_temp_readings,
       SUM(CASE WHEN humidity > 70 THEN 1 ELSE 0 END) AS high_humidity_readings
FROM raw_sensor_readings
GROUP BY location
ORDER BY location;

```

9.6 Anomaly Frequency Over Time

```

-- Anomalies grouped by 10-minute intervals
SELECT DATE_TRUNC('HOUR', detected_at) AS hour_bucket,
       metric_name,
       COUNT(*) AS alert_count
FROM anomaly_alerts
GROUP BY hour_bucket, metric_name
ORDER BY hour_bucket DESC;

```

Additional Resources

- [Spark Structured Streaming Programming Guide](#)
 - [Kafka Documentation](#)
 - [Snowflake Spark Connector](#)
 - [Window Operations on Event Time](#)
-